```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, roc_auc_score

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
```

```
df = pd.read_csv("heart.csv")  # Download your dataset and place in same folder
df.head()
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1

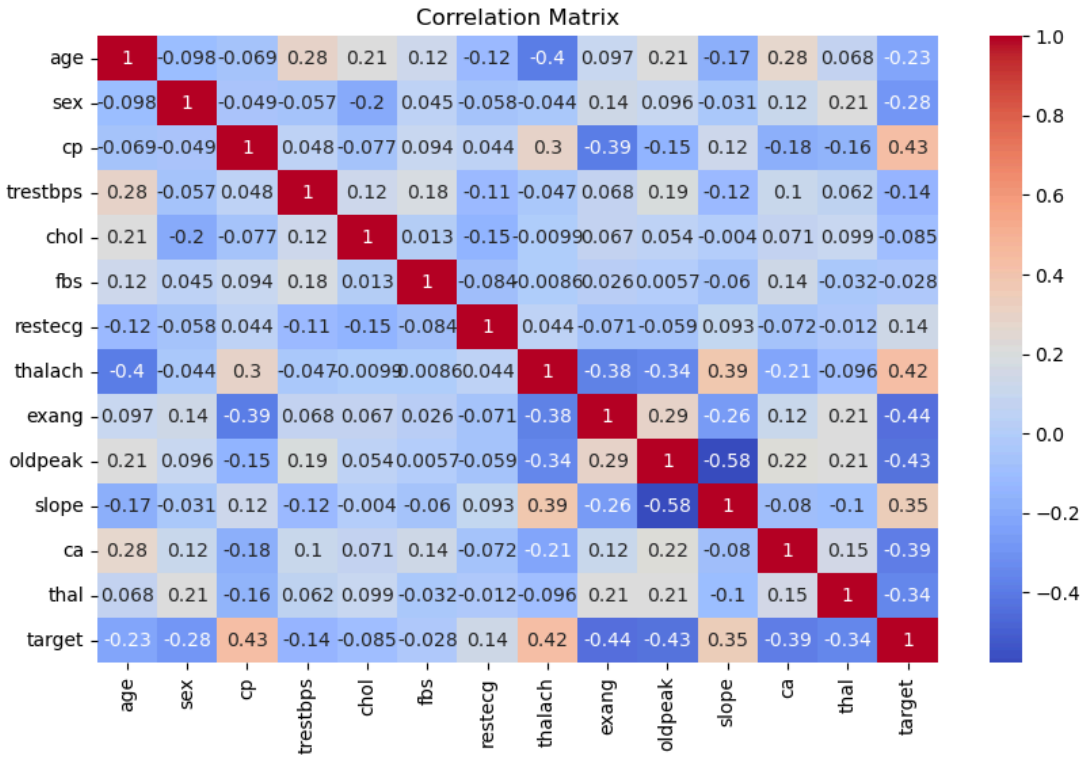
Start coding or generate with AI.

```
print(df.info())
print(df.isnull().sum())

# If missing values exist
df = df.dropna()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       303 non-null    int64
 1   sex       303 non-null    int64
 2   cp        303 non-null    int64
 3   trestbps  303 non-null    int64
 4   chol      303 non-null    int64
 5   fbs       303 non-null    int64
 6   restecg   303 non-null    int64
 7   thalach   303 non-null    int64
 8   exang     303 non-null    int64
 9   oldpeak   303 non-null    float64
 10  slope     303 non-null    int64
 11  ca        303 non-null    int64
 12  thal      303 non-null    int64
 13  target    303 non-null    int64
dtypes: float64(1), int64(13)
memory usage: 33.3 KB
None
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          0
thal        0
target      0
dtype: int64
```

```
plt.figure(figsize=(10,6))
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()
```

```
sns.countplot(x='target', data=df)
plt.title("Target Distribution")
plt.show()
```



```
from sklearn.metrics import accuracy_score

print("Logistic Regression Accuracy:", accuracy_score(y_test, lr_pred))
print("Decision Tree Accuracy:", accuracy_score(y_test, dt_pred))
```

```
Logistic Regression Accuracy: 0.8524590163934426
Decision Tree Accuracy: 0.8360655737704918
```

```
lr = LogisticRegression()
lr.fit(X_train, y_train)   # corrected here
lr_pred = lr.predict(X_test)

dt = DecisionTreeClassifier()
dt.fit(X_train, y_train)
dt_pred = dt.predict(X_test)

from sklearn.metrics import accuracy_score

print("Logistic Regression Accuracy:", accuracy_score(y_test, lr_pred))
print("Decision Tree Accuracy:", accuracy_score(y_test, dt_pred))
```

```
Logistic Regression Accuracy: 0.8524590163934426
Decision Tree Accuracy: 0.819672131147541
```

```
dt = DecisionTreeClassifier()
dt.fit(X_train, y_train)
dt_pred = dt.predict(X_test)
```

```
rf = RandomForestClassifier()
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)
```

```
def evaluate(model_name, y_test, y_pred):
    print(f"--- {model_name} ---")
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
    print("Classification Report:\n", classification_report(y_test, y_pred))
    print("ROC-AUC:", roc_auc_score(y_test, y_pred))
    print("\n")

evaluate("Logistic Regression", y_test, lr_pred)
evaluate("Decision Tree", y_test, dt_pred)
evaluate("Random Forest", y_test, rf_pred)
```

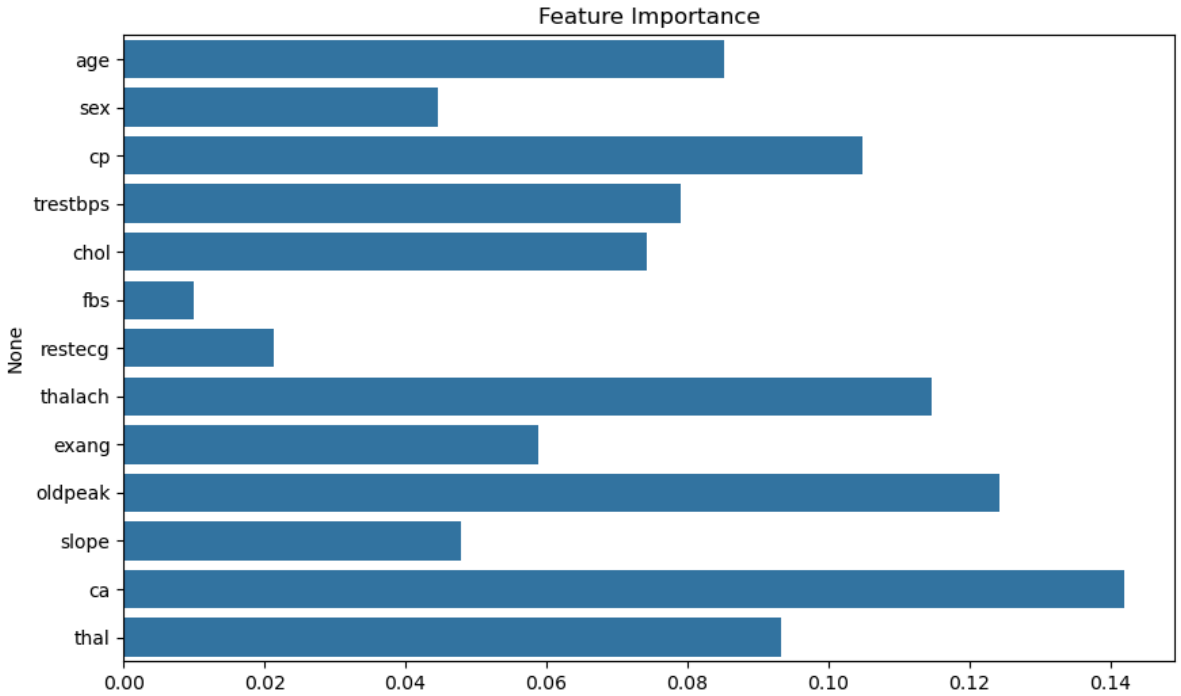
```
--- Logistic Regression ---
Accuracy: 0.8524590163934426
Confusion Matrix:
 [[25  4]
 [ 5 27]]
Classification Report:
               precision    recall  f1-score   support

           0       0.83      0.86      0.85        29
           1       0.87      0.84      0.86        32

    accuracy                           0.85        61
   macro avg       0.85      0.85      0.85        61
weighted avg       0.85      0.85      0.85        61

ROC-AUC: 0.8529094827586207


--- Decision Tree ---
Accuracy: 0.8360655737704918
Confusion Matrix:
 [[26  3]
 [ 7 25]]
Classification Report:
               precision    recall  f1-score   support

           0       0.79      0.90      0.84        29
           1       0.89      0.78      0.83        32

    accuracy                           0.84        61
   macro avg       0.84      0.84      0.84        61
weighted avg       0.84      0.84      0.84        61

ROC-AUC: 0.8389008620689655


--- Random Forest ---
Accuracy: 0.8360655737704918
Confusion Matrix:
 [[24  5]
 [ 5 27]]
Classification Report:
               precision    recall  f1-score   support

           0       0.83      0.83      0.83        29
           1       0.84      0.84      0.84        32

    accuracy                           0.84        61
   macro avg       0.84      0.84      0.84        61
weighted avg       0.84      0.84      0.84        61

ROC-AUC: 0.8356681034482758
```

```
params = {
    'n_estimators': [50, 100, 200],
    'max_depth': [3, 5, 10]
}
```

```
grid = GridSearchCV(RandomForestClassifier(), params, cv=5)
grid.fit(X_train, y_train)

print("Best Params:", grid.best_params_)
best_model = grid.best_estimator_
```

```
Best Params: {'max_depth': 3, 'n_estimators': 200}
```

```
importances = rf.feature_importances_
features = X.columns

plt.figure(figsize=(10,6))
sns.barplot(x=importances, y=features)
plt.title("Feature Importance")
plt.show()
```




```
sample = X_test[0].reshape(1, -1)
prediction = rf.predict(sample)

print("Heart Disease" if prediction[0] == 1 else "No Heart Disease")
```

```
No Heart Disease
```

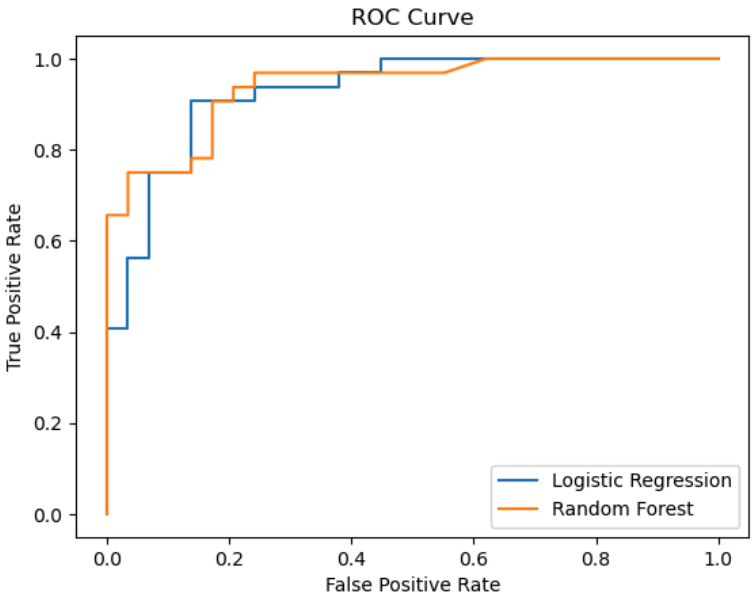
```
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier

# SVM
svm = SVC(probability=True)
svm.fit(X_train, y_train)
svm_pred = svm.predict(X_test)

# KNN
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
knn_pred = knn.predict(X_test)
```

```
models = {
    "Logistic Regression": lr_pred,
    "Decision Tree": dt_pred,
    "Random Forest": rf_pred,
    "SVM": svm_pred,
    "KNN": knn_pred
}

for name, pred in models.items():
    print(f"{name} Accuracy:", accuracy_score(y_test, pred))
```

```
Logistic Regression Accuracy: 0.8524590163934426
Decision Tree Accuracy: 0.8360655737704918
Random Forest Accuracy: 0.8360655737704918
```

```
SVM Accuracy: 0.8688524590163934
KNN Accuracy: 0.9016393442622951
```

```
from sklearn.metrics import roc_curve

lr_probs = lr.predict_proba(X_test)[:,1]
rf_probs = rf.predict_proba(X_test)[:,1]

lr_fpr, lr_tpr, _ = roc_curve(y_test, lr_probs)
rf_fpr, rf_tpr, _ = roc_curve(y_test, rf_probs)

import matplotlib.pyplot as plt

plt.plot(lr_fpr, lr_tpr, label="Logistic Regression")
plt.plot(rf_fpr, rf_tpr, label="Random Forest")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()
```




```
import seaborn as sns

cm = confusion_matrix(y_test, rf_pred)

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix - Random Forest")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

```
param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [4, 6, 10],
    'min_samples_split': [2, 5]
}

grid = GridSearchCV(RandomForestClassifier(), param_grid, cv=5)
grid.fit(X_train, y_train)

print("Best Parameters:", grid.best_params_)

best_rf = grid.best_estimator_
```

```
Best Parameters: {'max_depth': 10, 'min_samples_split': 2, 'n_estimators': 100}
```

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(rf, X, y, cv=5)
print("Cross Validation Scores:", scores)
print("Average Score:", scores.mean())
```

```
Cross Validation Scores: [0.85245902 0.85245902 0.81967213 0.76666667 0.75      ]
Average Score: 0.8082513661202185
```

```
import joblib

joblib.dump(rf, "heart_model.pkl")
print("Model Saved!")
```

```
Model Saved!
```

```
model = joblib.load("heart_model.pkl")

prediction = model.predict(X_test[0].reshape(1, -1))
print(prediction)
```

```
[0]
```

```
sample = X.iloc[0].values.reshape(1, -1)

sample = scaler.transform(sample)

if prediction[0] == 1:
    print("Patient likely has Heart Disease")
else:
    print("Patient likely does NOT have Heart Disease")
```

```
Patient likely does NOT have Heart Disease
```

Start coding or generate with AI.